Pwn

二进制漏洞利用:通过利用二进制程序的漏洞，达到控制程序行为的目的

Date: September 10, 2025

Press Space for next page →

本节内容

- 二进制程序漏洞
  - 整数溢出
  - 格式化字符串漏洞
  - **缓冲区溢出漏洞**
    - 堆、栈
  - UAF、double-free
- 控制流劫持技术
  - 篡改程序的正常执行流，来执行恶意代码
    - 执行恶意的`shellcode`或代码复用攻击
  - 代码注入攻击
  - 代码重用攻击
    - **ROP返回式编程**
    - JOP面向跳转的编程
- 防御技术

1. 栈溢出漏洞
2. 函数调用栈帧变化举例
3. 调用约定cdecl
4. 控制流劫持技术：ROP
5. 防御技术
6. 实验报告

栈溢出漏洞

栈：从高地址向低地址、先进后出

- 📝 **What's** - 程序向栈中某个变量中写入的字节数超过了这个变量本身所申请的字节数，因而导致与其相邻的栈中的变量的值被改变。
  - 这是一种特定的缓冲区溢出漏洞
  - 类似的有堆溢出，.bss段溢出

- 函数调用–栈帧
  - 局部变量、返回地址、参数

- 🤹问题
  - 如果局部变量是一个数组，会发生什么？

```
void myFunction(int x, int y) {
    int arr[3] = {10, 20, 30};
    // 其他代码
}
```

```
栈的高地址（例如 0x7fffd4b0）
+-------------------+
|    (函数参数y)    | <- 参数 y (值 20)
+-------------------+
|    (函数参数x)    | <- 参数 x (值 10)
+-------------------+
|    (返回地址)     | <- 返回地址
+-------------------+
|    (保存的旧rbp)  | <- 上一个栈帧的基址指针 (saved base pointer)
+-------------------+
|    (局部变量 arr[2]) | <- arr[2] (值 30)
+-------------------+
|    (局部变量 arr[1]) | <- arr[1] (值 20)
+-------------------+
|    (局部变量 arr[0]) | <- arr[0] (值 10)
+-------------------+
|    (其他局部变量) | <- 其他局部变量，可能存在
+-------------------+
|    (栈帧填充)     | <- 填充区域，保证内存对齐
+-------------------+
栈的低地址
```

函数调用栈帧变化举例

```c
# include<stdio.h>
# include<stdlib.h>

int fun(int a, int b, int c){
    int d;
    d = a+b+c;
    return d;
}

int main(){
    fun(1,2,3);
    exit(0);
    return 0;
}
```

`gcc call.c -m32 -S -o call.s` 生成32位代码的汇编文件 👉👉

```
08049196 <fun>:
 8049196: f3 0f 1e fb          endbr32
 804919a: 55                   push
 804919b: 89 e5                mov
 804919d: 83 ec 10             sub
 80491a0: e8 50 00 00 00       call
 80491a5: 05 5b 2e 00 00       add
 80491aa: 8b 55 08             mov
 80491ad: 8b 45 0c             mov
 80491b0: 01 c2                add
 80491b2: 8b 45 10             mov
 80491b5: 01 d0                add
 80491b7: 89 45 fc             mov
 80491ba: 8b 45 fc             mov
 80491bd: c9                   leave
 80491be: c3                   ret
```

```
080491bf <main>:
 80491bf: f3 0f 1e fb          endbr32
 80491c3: 8d 4c 24 04          lea
 80491c7: 83 e4 f0             and
 80491ca: ff 71 fc             push
 80491cd: 55                   push
 80491ce: 89 e5                mov
 80491d0: 53                   push
 80491d1: 51                   push
 80491d2: e8 f9 fe ff ff       call
 80491d7: 81 c3 29 2e 00 00    add
 80491dd: 6a 03                push
 80491df: 6a 02                push
 80491e1: 6a 01                push
 80491e3: e8 ae ff ff ff       call
 80491e8: 83 c4 0c             add
 80491eb: 83 ec 0c             sub
 80491ee: 6a 00                push
 80491f0: e8 6b fe ff ff       call
```

```
080491bf <main>:
 80491bf: f3 0f 1e fb           endbr32
 80491c3: 8d 4c 24 04           lea    ecx,[esp+0x4]
 80491c7: 83 e4 f0              and    esp,0xfffffff0 //16字节对齐
 80491ca: ff 71 fc              push   DWORD PTR [ecx-0x4]
 //main()返回地址入栈
 80491cd: 55                    push   ebp //保存旧的ebp
 80491ce: 89 e5                 mov    ebp,esp //设置当前ebp
 80491d0: 53                    push   ebx
 80491d1: 51                    push   ecx
 80491d2: e8 f9 fe ff ff        call   80490d0 <__x86.get_pc_thunk.bx>
 // 获取当前PC，并存储在bx
 80491d7: 81 c3 29 2e 00 00     add    ebx,0x2e29

 80491dd: 6a 03                 push   0x3 // 参数入栈
 80491df: 6a 02                 push   0x2
 80491e1: 6a 01                 push   0x1
 80491e3: e8 ae ff ff ff        call   8049196 <fun>
 80491e8: 83 c4 0c              add    esp,0xc //清理栈参数
 80491eb: 83 ec 0c              sub    esp,0xc //调整栈空间
 80491ee: 6a 00                 push   0x0
 80491f0: e8 6b fe ff ff        call   8049060 <exit@plt>
```

1. 栈空间分配及初始化

```
-------高地址----------
      旧的ebp           <-- esp,ebp
       ebx
       ecx
-------低地址----------
```

2. 栈空间布局

```
-------高地址----------
      旧的ebp           <-- ebp
       ebx
       ecx
     参数3 (0x3)
     参数2 (0x2)
     参数1 (0x1)       <-- esp
-------低地址----------
```

3. `call 8049196 <fun>`

```
push eip
jmp 8049196 <fun>
```

1. 被调用者callee

```
-------高地址----------
      旧的ebp       <-- ebp
       ebx
       ecx
     参数3 (0x3)
     参数2 (0x2)
     参数1 (0x1)
      0x80491e8     <-- esp
-----------------
```

```
-----------------
    main()的ebp       <-- esp,ebp
```

```
-----------------
-
-
-
-                        <-- esp
```

```
08049196 <fun>:
 8049196: f3 0f 1e fb          endbr32
 804919a: 55                   push   ebp
 804919b: 89 e5                mov    ebp,esp

 804919d: 83 ec 10             sub    esp,0x10
 80491a0: e8 50 00 00 00       call   80491f5 <__x86.get_pc_thunk.ax>
 80491a5: 05 5b 2e 00 00       add    eax,0x2e5b
 80491aa: 8b 55 08             mov    edx,DWORD PTR [ebp+0x8]
 80491ad: 8b 45 0c             mov    eax,DWORD PTR [ebp+0xc]
 80491b0: 01 c2                add    edx,eax
 80491b2: 8b 45 10             mov    eax,DWORD PTR [ebp+0x10]
 80491b5: 01 d0                add    eax,edx
 80491b7: 89 45 fc             mov    DWORD PTR [ebp-0x4],eax
 80491ba: 8b 45 fc             mov    eax,DWORD PTR [ebp-0x4]
 80491bd: c9                   leave
 80491be: c3                   ret
```

1. `leave` == `mov esp, ebp`、`pop ebp`
2. `ret` == `pop eip`

2. 调用者caller

```
-------高地址----------
      旧的ebp      <-- ebp
       ebx
       ecx
     参数3 (0x3)
     参数2 (0x2)
     参数1 (0x1)   <-- esp
-----------------
```

```
080491bf <main>:
 ...
 ...
 80491e3: e8 ae ff ff ff        call   8049196 <fun>
 80491e8: 83 c4 0c              add    esp,0xc //清理栈参数
 80491eb: 83 ec 0c              sub    esp,0xc //调整栈空间
 80491ee: 6a 00                 push   0x0
 80491f0: e8 6b fe ff ff        call   8049060 <exit@plt>
```

调用约定

- What's：约定函数之间如何传递参数、约定函数如何传递返回值
- x86常见的调用约定
  - `caller clean-up`
    - `cdecl` (C declaration)：参数从右到左入栈
  - `callee clean-up`
    - `stdcall` (参数从右到左入栈)、`fastcall` （前两个参数使用寄存器传递`ecx`、`edx`）

调用约定cdecl

- 32位 cdecl
  - 用栈传递参数、用寄存器`eax`保存返回值
- 64位 cdecl
  - 用`rdi` `rsi` `rdx` `rcx` `r8` `r9`传递前6个参数
  - 剩下的参数用栈传递
  - 用寄存器`rax`保存返回值
- 栈顶指针`ebp` `rbp`的用途
  - 索引栈上的参数
  - 保存栈顶`esp` `rsp`位置

控制流劫持技术：ROP

Return Oriented Programming

- 漏洞利用简单形式：代码注入攻击-执行shellcode
- 防御技术：DEP数据执行保护

> **DEP** 限定内存页不能同时具有执行权限和写权限

- 进阶：**代码重用攻击-ROP**,绕过**DEP**
- 📝 **核心思想**
  - 栈缓冲区溢出的基础上，利用程序中已有的小片段（`gadgets`），构造ROP链，来改变某些寄存器或者变量的值，从而控制程序的执行流程.

- 🤹 **基本类型**
  - ret2text
  - ret2shellcode
  - ret2syscall
  - ret2libc

【参考资料】栈溢出与ROP

> Gadget:以 `ret` 指令为结尾的代码片段（程序中合法的代码片段）
> ROP链：精心设计的gadgets

gadgets

- 🛠️ **gadgets** 通俗理解
  - 利用 `pop reg; ret` 的特性，给寄存器赋值，进而满足函数调用的条件
  - 通常可以用 `p64(pop_rdi) + p64(num)` 等片段给寄存器赋值
- 举个🌰
  - `p64(pop_rdi)+p64(0)+ p64(pop_rsi)+p64(buf_addr) + p64(pop_rdx) + p64(0x100) + p64(read_plt)`
  - `read(0,buf,0x100);`
- `one gadget`
  - 一个特殊的 `gadget`，它在执行时，能够直接生成一个命令行 `shell`，通常无需再链接其他 `gadget`
  - 可以直接getshell的gadget
  - 使用工具 `one_gadget` 扫描和查找 `libc` 库

防御技术

NX

- 基本原理：将数据所在的内存页标记为不可执行
- 当程序溢出转为`shellcode`时，程序尝试在数据页执行指令，CPU直接抛异常
- gcc编译默认开启NX

DEP

栈Canary

> 栈保护检查，函数执行时先在栈里插入cookie，函数返回时验证cookie是否被篡改

Windows平台下canary的编译支持与检验

- 微软MSVC编译器开启`/GS`（Stack Guard）选项
- 地址`FS:[0x28]`（FS：段寄存器）：表示当前线程的`stack cookie`，即canary
  - `__readfsqword(0x28u)`：Windows平台下读取canary的方法

```
// 贴近编译器生成的伪代码
unsigned __int64 callme()
{
    unsigned __int64 __stack_canary = __readfsqword(0x28u);// 双下划线开头的变量，属于编译器变量
    // 函数主体
    // ...
    if (__readfsqword(0x28u) != __stack_canary)
    {
        __security_check_cookie_fail();  // Canary 值被篡改，可能存在栈溢出
    }
    return 0;  // 正常退出
}
```

段寄存器

> 用于分段内存管理，段寄存器的值+偏移量组合成内存单元的**物理地址**
> 在早期16位实模式中很关键
> 在32位保护模式、64位长模式中主要是 `FS` 、 `GS` 用于 `TLS` 线程管理

- 实模式下，`物理地址 = 段寄存器 * 16 + 偏移量`
  - 如`[ds:0x1234]`表示地址`DS × 16 + 0x1234`

`CS`	Code Segment	**代码段寄存器**，指示代码所在的段地址（与 `IP` 配合）
`DS`	Data Segment	**数据段寄存器**，默认用于访问数据（变量、数组）
`SS`	Stack Segment	**栈段寄存器**，与 `SP/ESP` 配合访问栈
`ES`	Extra Segment	**额外段寄存器**，字符串操作和扩展用途
`FS`	Additional Segment	**附加数据段寄存器**，操作系统/线程局部存储使用
`GS`	General Segment	**一般段寄存器**，和 `FS` 类似

Linux平台下canary的编译支持与检验

- GCC编译器指定 `gcc -fstack-protector`
  - 不开启栈保护：`gcc -fno-stackc-protector`
- `glibc` 使用符号变量 `__stack_chk_guard` 作为当前线程的canary。
- 检验：

```
$ gcc -fstack-protector-all -o test test.c
$ objdump -d test | grep __stack_chk_guard
```

- x86-64下， `glibc` 将 `__stack_chk_guard` 实现为 `TLS` 变量,保存在 `fs:0x28`

```
mov    rax,QWORD PTR fs:0x28
mov    QWORD PTR [rbp-0x8],rax      ; 保存 canary 到局部变量

...

mov    rax,QWORD PTR fs:0x28
xor    rax,QWORD PTR [rbp-0x8]
je     .Lreturn_ok
call   __stack_chk_fail             ; Canary 被破坏，执行终止
```

ASLR

- 栈、共享库、堆地址都会被随机化
- `/proc/sys/kernel/randomize_va_space = 0` 无随机化

> 开启ASLR后，栈、共享库、堆地址都会被随机化，但是程序本身代码通常固定加载在 0x8048000 编译

PIE 地址无关代码

- 在编译时生成地址无关代码（PIE），则程序可以加载在任意地址执行

```
# 在OS开启ASLR的前提下，编译程序时指定开启pie
gcc -fPIE -pie helloworld.c -o with_pie

gcc -no-pie helloworld.c -o no_pie
```

- 查看程序是否开启 `pie`：
  - `file`、`readelf`

基于硬件–控制流强制技术 CET

> Control-flow Enforcement Technology

- 基于硬件的
- 预防控制流劫持攻击
  - 包括前向控制流劫持(JMP CALL)
  - 后向控制流劫持（RET）

> 目前，64位内核中，仅支持**用户空间**的 `影子栈` 技术和**内核空间**的 `IBT` 技术

Shadow Stack-影子栈

> 在内存中分配第二个栈。影子栈是不能被应用直接修改的。 在执行函数调用（ CALL 指令）时，
> 处理器将 返回地址 同时 push 到 原栈 和 影子栈 上。
> 等函数返回时，比较 影子栈 的 返回地址 与 原栈 是否一致。
> 若不同，则触发 control protection fault

- `影子栈`的实质是映射一块**私有、匿名、只读**的内存
  - 私有。相对：共享
  - 匿名页面。相对：文件页面，即映射到文件的页面。
  - 只读：影子栈只读，只用来存储返回地址。

IBT(Indirect-branch-tracking)-间接分支追踪

0. 间接分支
   - 依赖动态计算目标地址
   - 包括函数调用`call`、跳转`jmp`、`RET`
   - 目标地址通常保存在指针里

1. 间接分支追踪

> 验证间接调用(CALL)或跳转（ JMP ）的目标是否使用 endbr 操作码标记。

- `endbr64`用于64位；
- `endbr32`用于32位
- 用于间接分支保护。编译器在每个可间接调用的函数入口插入`endbr64`指令。如果`jmp``call`跳转到非`endbr`开头的函数，则CPU触发异常。

实验报告

- 命名方式：学号-姓名-PWN.pdf
- 截止日期：夏季学期**第四周周五晚8点**前
- 要求：
  - 应包括相关知识点简要总结、思路分析、**exp代码（含注释）**、及实验思考（如果有的话）
  - **补充要求：内容精简，不要有废话、套话**